



UNIVERSIDAD SERGIO ARBOLEDA

**DESEMPEÑO Y COMPARACIÓN PSO VS. ACO
PARA EL PROBLEMA DEL VIAJANTE (TSP)**

Inteligencia Artificial

Camilo Andrés Díaz García

*Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá D.C., 2026*

Índice

1. Introducción	2
2. Metodología de PSO	2
3. Resultados por instancia	2
4. Comparación PSO vs. ACO	3
4.1. Diseño del barrido	3
4.2. Por qué PSO pesa	5
4.3. Rutas	5
4.4. Conclusiones de la comparación	6
5. Conclusiones	7
A. Resultados completos del barrido	8

1 Introducción

Este trabajo implementa PSO (*Particle Swarm Optimization*) para el TSP mediante codificación de claves aleatorias, en las mismas tres instancias del proyecto ACO-IA ($n=20$, 2 000 y 200 000 ciudades), midiendo tiempo y memoria, y comparando los resultados directamente contra las cifras ya obtenidas con ACO en ese proyecto.

2 Metodología de PSO

Cada partícula tiene un vector de posición continuo $\mathbf{x}_i \in \mathbb{R}^n$ (una clave por ciudad). Para evaluarla, se ordenan las ciudades por el valor de su clave (*argsort*); ese orden es el tour, evaluado con la misma función objetivo de ACO ($L(\text{tour}) = \sum d(c_i, c_{i+1}) + d(c_n, c_1)$, distancia euclidiana). La actualización es la estándar:

$$\mathbf{v}_i(t+1) = w \mathbf{v}_i(t) + c_1 r_1 [\mathbf{p}_i - \mathbf{x}_i(t)] + c_2 r_2 [\mathbf{g} - \mathbf{x}_i(t)], \quad (1)$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1), \quad (2)$$

con w (inercia), c_1 (cognitivo), c_2 (social) y $r_1, r_2 \sim U(0, 1)$ independientes por componente. $\mathbf{p}_i$ es la mejor posición personal de la partícula i y $\mathbf{g}$ la mejor global del enjambre. Las claves se recortan a $[-50, 50]$ (con reinicio de velocidad a 0 al recortar) si una componente se dispara; no hay más tratamiento de límites porque las claves solo importan por su orden relativo.

Parámetros usados en las tres instancias: $w = 0.7$, $c_1 = c_2 = 1.5$, inicialización $\mathbf{x}_i \sim U(0, 1)^n$, $\mathbf{v}_i(0) = \mathbf{0}$. El número de partículas e iteraciones se ajustó por instancia (partículas moderadas, más iteraciones, no un enjambre gigantesco) y se documenta en la Tabla 1. Implementado en C++20 con OpenMP, paralelizando la evaluación de partículas por hilo con un buffer reutilizado (sin reasignar memoria en el bucle caliente), igual que en ACO.

3 Resultados por instancia

Tabla 1: Resultados de PSO por instancia (semillas 1–3 para $n=2\,000$ y $n=200\,000$; 1–5 para $n=20$).

Instancia	Partículas	Iteraciones	L_{mejor} media	Tiempo medio (s)	Memoria pico (MB)
$n = 20$	200	300	4,344 (desv. 0,576)	0,131	4,5
$n = 2\,000$	200	500	978,627 (desv. 19,578)	2,264	9,4
$n = 200\,000$	100	100	103 879,712 (desv. 73,322)	41,606	266,3

En $n = 20$, PSO se validó contra el óptimo exacto de Held-Karp (calculado en ACO-IA): con 200 partículas y 300 iteraciones, 3 de 5 semillas llegan a un gap menor al 1 %, pero 2 de

5 quedan entre 11 % y 23 % — la calidad depende fuertemente de la semilla, a diferencia de ACO. En $n = 2000$ y $n = 200000$ no hay óptimo exacto disponible; la comparación relevante es contra ACO (sección 4). En las tres instancias todos los tours se verificaron como permutaciones válidas.

4 Comparación PSO vs. ACO

4.1 Diseño del barrido

Agentes = partículas (PSO) = hormigas (ACO). Para cada $n \in \{20, 200, 2000, 20000\}$: agentes $\in \{50, 500, 2048\}$ e iteraciones $\in \{5, 20, 100\}$ (9 configuraciones), con las semillas 1–3; en $n = 200000$ solo 2 048 agentes con 5 y 20 iteraciones. Ambos algoritmos leen el mismo archivo de ciudades por $(n, \text{semilla})$. ACO usa los parámetros oficiales de ACO-IA ($K = 8$, $K = 19$ en $n = 20$; $\alpha = 1,5$, $\beta = 5$, $\rho = 0,1$, $qfac=3$, segunda feromona con $\alpha_2 = 1,0$) y PSO los fijos de este trabajo ($w = 0,7$, $c_1 = c_2 = 1,5$), sin ajustarlos. El tiempo es preparación más algoritmo. L_{ref} es el óptimo de Held-Karp en $n = 20$ y, en los demás n , la mejor longitud medida para esa instancia (en $n = 200000$, la mejor de todas las corridas). La fila de $n = 200000$ con 108 iteraciones proviene de la ronda anterior (semilla 1, ciudades generadas internamente por cada programa, no por archivo; las otras filas están en resultados/barrido.csv). No se corrió $P = 20000$ en $n = 200000$: la fórmula de memoria de PSO (abajo) da ≈ 48 GB, frente a 15,88 GB de RAM.

Tabla 2: ACO vs. PSO con 2 048 agentes (media $\pm$ desv. de 3 semillas; $n = 200000$ con 108 it.: semilla 1). Tabla completa en el apéndice.

n	Agentes	It.	L_{ACO}	L_{PSO}	t_{ACO} (s)	t_{PSO} (s)	Mem. ACO (MB)	Mem. PSO (MB)
20	2 048	5	4,535 $\pm$ 0,359	6,993 $\pm$ 0,372	0,01	0,01	4,6	5,0
20	2 048	20	4,535 $\pm$ 0,359	5,401 $\pm$ 1,002	0,03	0,01	4,5	5,0
20	2 048	100	4,535 $\pm$ 0,359	4,768 $\pm$ 0,575	0,17	0,07	4,5	5,0
200	2 048	5	11,712 $\pm$ 0,130	90,261 $\pm$ 1,111	0,04	0,02	4,6	9,2
200	2 048	20	11,526 $\pm$ 0,083	87,975 $\pm$ 2,186	0,14	0,06	4,6	9,2
200	2 048	100	11,423 $\pm$ 0,178	82,645 $\pm$ 4,606	0,67	0,32	4,6	9,2
2 000	2 048	5	38,605 $\pm$ 0,118	1 007,924 $\pm$ 7,987	0,35	0,17	5,6	51,7
2 000	2 048	20	38,018 $\pm$ 0,469	1 005,522 $\pm$ 6,900	1,36	0,69	5,6	51,7
2 000	2 048	100	37,452 $\pm$ 0,213	1 000,979 $\pm$ 3,988	6,69	3,53	5,6	51,7
20 000	2 048	5	123,380 $\pm$ 0,313	10 303,629 $\pm$ 42,434	3,79	2,16	14,3	476,6
20 000	2 048	20	121,808 $\pm$ 0,407	10 287,972 $\pm$ 31,052	14,57	8,56	14,3	476,6
20 000	2 048	100	121,162 $\pm$ 0,462	10 274,472 $\pm$ 42,107	71,15	42,38	14,3	476,6
200 000	2 048	5	390,059 $\pm$ 0,512	103 803,161 $\pm$ 49,981	45,29	27,30	102,4	4 726,1
200 000	2 048	20	386,061 $\pm$ 0,335	103 787,889 $\pm$ 60,275	176,99	108,27	102,5	4 726,0
200 000	2 048	108	385,479	103 706,095	961,70	610,08	102,8	4 724,7

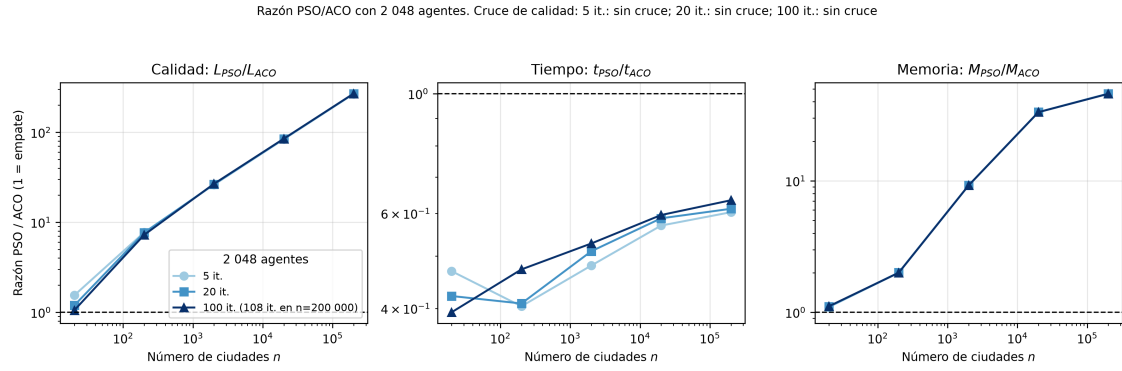


Figura 1: Razón PSO/ACO de calidad, tiempo y memoria en función de n , con 2 048 agentes. La línea punteada es la razón 1. En calidad y memoria la razón de PSO crece con n sin cruzar el 1; en tiempo se mantiene por debajo de 1.

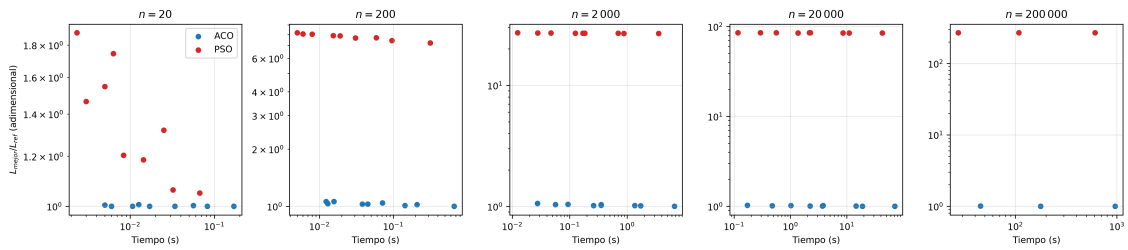


Figura 2: Calidad (L_{mejor}/L_{ref}) contra tiempo para las 9 configuraciones de cada n (2 o 3 en $n = 200\,000$; una de ellas con 108 it.). Con el mismo tiempo ACO queda más cerca de L_{ref} .

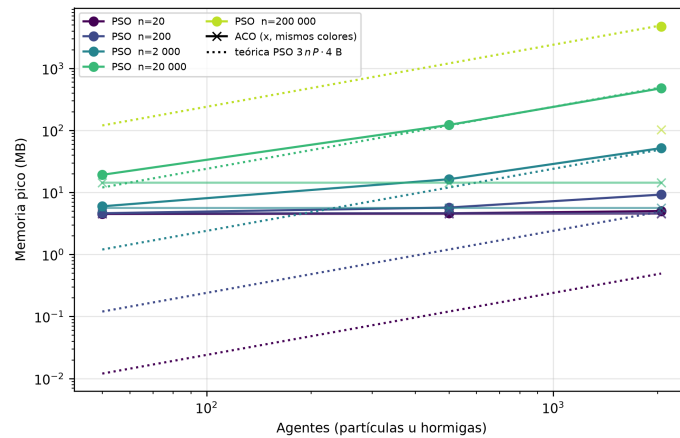


Figura 3: Memoria pico contra agentes. Línea punteada: $3nP \cdot 4$ bytes (PSO).

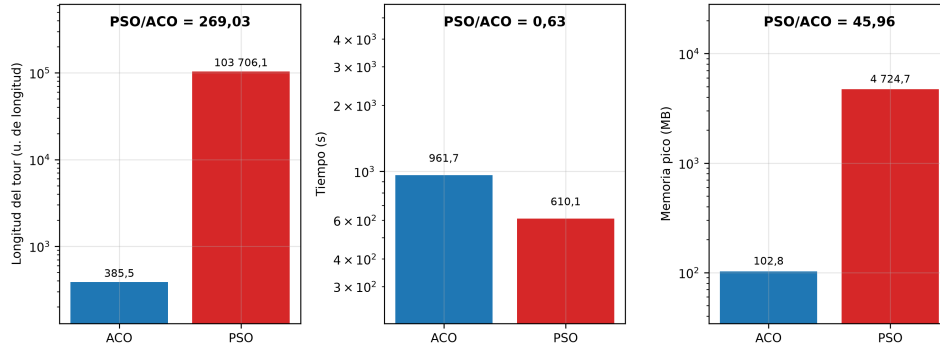


Figura 4: $n = 200\,000$, $P = m = 2048$, 108 iteraciones: longitud, tiempo y memoria (escala logarítmica).

4.2 Por qué PSO pesa

Cada partícula guarda tres vectores de tamaño n (posición, velocidad y mejor personal), así que la memoria crece como $3nP \cdot 4$ bytes (Figura 3). En PSO con 2048 partículas la memoria pico medida dividida por $3nP \cdot 4$ bytes vale 10,17 ($n = 20$), 1,87 ($n = 200$), 1,05 ($n = 2000$), 0,97 ($n = 20000$), 0,96 ($n = 200000$); para $n \geq 2000$ la fórmula describe lo medido, y en n pequeña domina la memoria base del proceso. Además, cada evaluación exige ordenar n claves, $O(n \log n)$, y cada iteración sortea $2n$ números aleatorios por partícula. ACO solo mantiene feromona sobre $n \cdot K$ aristas y no guarda estado por hormiga entre iteraciones.

4.3 Rutas

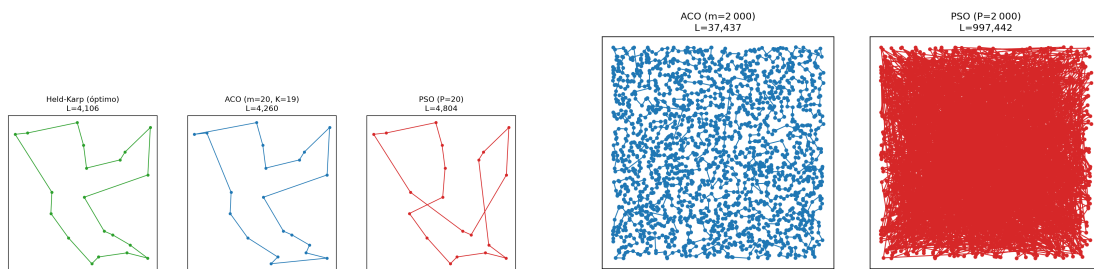


Figura 5: Rutas en $n = 20$ (Held-Karp, ACO, PSO) y $n = 2000$ (ACO, PSO), ronda anterior.

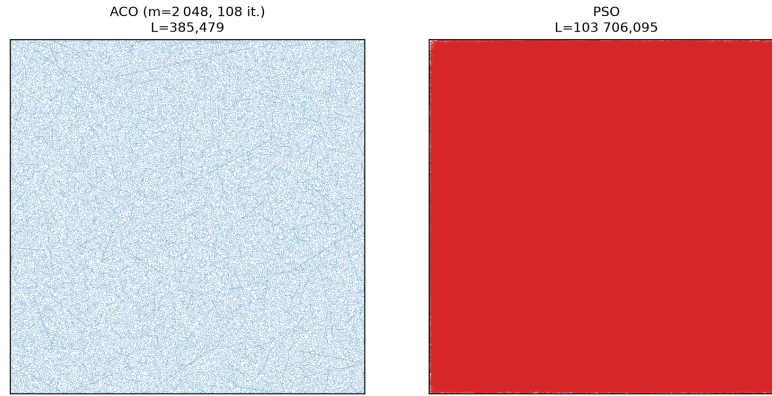


Figura 6: Rutas en $n = 200\,000$ (ACO y PSO, $P = m = 2\,048$, 108 it.).

4.4 Conclusiones de la comparación

- **Calidad: ACO es mejor.** Con 2 048 agentes ACO obtiene menor L_{mejor} en 15 de 15 combinaciones de n y presupuesto; en las 9 configuraciones emparejadas de cada $n \leq 20\,000$ gana en 9/9 ($n = 20$), 9/9 ($n = 200$), 9/9 ($n = 2\,000$), 9/9 ($n = 20\,000$). Razón mediana L_{PSO}/L_{ACO} : $n = 20$: 1,32, $n = 200$: 7,63, $n = 2\,000$: 26,33, $n = 20\,000$: 84,42.
- **Dependencia de n : no hay cruce en el rango probado ($n = 20$ a $n = 200\,000$).** Razón L_{PSO}/L_{ACO} con 2 048 agentes y 100 it. (108 en $n = 200\,000$): $n = 20$: 1,05, $n = 200$: 7,23, $n = 2\,000$: 26,73, $n = 20\,000$: 84,80, $n = 200\,000$: 269,03.
- **Presupuesto.** Pasar de 5 a 100 iteraciones (de 5 a 20 en $n = 200\,000$) mejora L_{mejor} , con 2 048 agentes, en ACO 0,0 % ($n = 20$), 2,5 % ($n = 200$), 3,0 % ($n = 2\,000$), 1,8 % ($n = 20\,000$), 1,0 % ($n = 200\,000$) y en PSO 31,8 % ($n = 20$), 8,4 % ($n = 200$), 0,7 % ($n = 2\,000$), 0,3 % ($n = 20\,000$), 0,0 % ($n = 200\,000$).
- **Tiempo: PSO es más rápido en todas las configuraciones emparejadas.** Razón t_{PSO}/t_{ACO} mediana en las 9 configuraciones: 0,47 (rango 0,39–0,50, PSO más rápido en 9/9; $n = 20$), 0,43 (rango 0,40–0,51, PSO más rápido en 9/9; $n = 200$), 0,50 (rango 0,45–0,53, PSO más rápido en 9/9; $n = 2\,000$), 0,60 (rango 0,55–0,68, PSO más rápido en 9/9; $n = 20\,000$); con $n = 200\,000$, 2 048 agentes y 108 it.: 0,63.
- **Memoria: PSO usa más en todo el rango.** Razón M_{PSO}/M_{ACO} con 2 048 agentes: 1,1 ($n = 20$), 2,0 ($n = 200$), 9,2 ($n = 2\,000$), 33,3 ($n = 20\,000$), 46,2 ($n = 200\,000$).
- **A igual tiempo:** $n = 20$ ($T = 0,07$ s): L/L_{ref} ACO 1,000, PSO 1,050; $n = 200$ ($T = 0,32$ s): L/L_{ref} ACO 1,009, PSO 7,236; $n = 2\,000$ ($T = 3,53$ s): L/L_{ref} ACO 1,010, PSO 26,806; $n = 20\,000$ ($T = 42,38$ s): L/L_{ref} ACO 1,001, PSO 84,844; $n = 200\,000$ ($T = 610,08$ s): L/L_{ref} ACO 1,002, PSO 269,031.

- *Límites:* PSO con parámetros fijos ($w = 0,7$, $c_1 = c_2 = 1,5$) y sin búsqueda local; los resultados valen para esta implementación.

5 Conclusiones

1. PSO con claves aleatorias converge en instancias pequeñas ($n = 20$), pero su calidad relativa a ACO empeora al crecer n , sin cruce en el rango probado ($n = 20$ a $n = 200000$).
2. PSO es más rápido por corrida en todas las configuraciones emparejadas, pero usa mucha más memoria, que crece como $3nP \cdot 4$ bytes; con el mismo tiempo, ACO llega a mejor calidad (sección 4).
3. Para TSP a esta escala, ACO (lista de K candidatas y feromona por arista) es la opción superior; PSO con claves aleatorias, parámetros fijos y sin búsqueda local no está adaptado a este problema, y los resultados valen para esta implementación.

A Resultados completos del barrido

Tabla 3: ACO vs. PSO, 9 configuraciones por n (media $\pm$ desv. de 3 semillas).

n	Agentes	It.	L ACO	L PSO	t ACO (s)	t PSO (s)	Mem. ACO (MB)	Mem. PSO (MB)
20	50	5	4,556 $\pm$ 0,378	8,512 $\pm$ 0,519	0,01	0,00	4,5	4,5
20	50	20	4,563 $\pm$ 0,348	7,884 $\pm$ 0,087	0,01	0,01	4,5	4,5
20	50	100	4,549 $\pm$ 0,372	5,987 $\pm$ 0,596	0,06	0,02	4,5	4,5
20	500	5	4,535 $\pm$ 0,359	6,616 $\pm$ 0,154	0,01	0,00	4,5	4,6
20	500	20	4,535 $\pm$ 0,359	5,448 $\pm$ 0,160	0,02	0,01	4,5	4,6
20	500	100	4,535 $\pm$ 0,359	4,822 $\pm$ 0,580	0,08	0,03	4,5	4,6
20	2048	5	4,535 $\pm$ 0,359	6,993 $\pm$ 0,372	0,01	0,01	4,6	5,0
20	2048	20	4,535 $\pm$ 0,359	5,401 $\pm$ 1,002	0,03	0,01	4,5	5,0
20	2048	100	4,535 $\pm$ 0,359	4,768 $\pm$ 0,575	0,17	0,07	4,5	5,0
200	50	5	12,087 $\pm$ 0,105	93,352 $\pm$ 1,640	0,01	0,01	4,6	4,6
200	50	20	12,090 $\pm$ 0,156	91,779 $\pm$ 0,527	0,02	0,01	4,6	4,6
200	50	100	11,907 $\pm$ 0,123	87,767 $\pm$ 1,508	0,07	0,03	4,6	4,6
200	500	5	11,783 $\pm$ 0,231	91,934 $\pm$ 1,887	0,01	0,01	4,6	5,7
200	500	20	11,714 $\pm$ 0,110	89,902 $\pm$ 1,545	0,05	0,02	4,6	5,7
200	500	100	11,669 $\pm$ 0,071	85,006 $\pm$ 4,643	0,21	0,10	4,6	5,7
200	2048	5	11,712 $\pm$ 0,130	90,261 $\pm$ 1,111	0,04	0,02	4,6	9,2
200	2048	20	11,526 $\pm$ 0,083	87,975 $\pm$ 2,186	0,14	0,06	4,6	9,2
200	2048	100	11,423 $\pm$ 0,178	82,645 $\pm$ 4,606	0,67	0,32	4,6	9,2
2000	50	5	39,539 $\pm$ 0,249	1 014,505 $\pm$ 11,780	0,03	0,01	5,6	5,9
2000	50	20	38,669 $\pm$ 0,040	1 009,993 $\pm$ 8,551	0,06	0,03	5,6	6,0
2000	50	100	38,021 $\pm$ 0,033	1 004,623 $\pm$ 8,546	0,26	0,13	5,6	6,0
2000	500	5	38,893 $\pm$ 0,064	1 008,236 $\pm$ 7,713	0,09	0,05	5,6	16,3
2000	500	20	38,219 $\pm$ 0,255	1 006,244 $\pm$ 4,922	0,35	0,18	5,6	16,3
2000	500	100	37,719 $\pm$ 0,410	1 002,152 $\pm$ 5,770	1,72	0,88	5,6	16,3
2000	2048	5	38,605 $\pm$ 0,118	1 007,924 $\pm$ 7,987	0,35	0,17	5,6	51,7
2000	2048	20	38,018 $\pm$ 0,469	1 005,522 $\pm$ 6,900	1,36	0,69	5,6	51,7
2000	2048	100	37,452 $\pm$ 0,213	1 000,979 $\pm$ 3,988	6,69	3,53	5,6	51,7
20000	50	5	123,894 $\pm$ 0,135	10 326,994 $\pm$ 41,975	0,17	0,12	14,3	19,2
20000	50	20	122,518 $\pm$ 0,399	10 322,464 $\pm$ 41,156	0,47	0,29	14,3	19,2
20000	50	100	121,663 $\pm$ 0,617	10 300,515 $\pm$ 20,671	2,23	1,37	14,3	19,2
20000	500	5	123,386 $\pm$ 0,545	10 315,746 $\pm$ 30,913	1,01	0,56	14,3	122,2
20000	500	20	122,076 $\pm$ 0,604	10 306,164 $\pm$ 37,608	3,68	2,24	14,3	122,2
20000	500	100	121,231 $\pm$ 0,260	10 298,055 $\pm$ 30,671	18,97	10,96	14,3	122,2
20000	2048	5	123,380 $\pm$ 0,313	10 303,629 $\pm$ 42,434	3,79	2,16	14,3	476,6
20000	2048	20	121,808 $\pm$ 0,407	10 287,972 $\pm$ 31,052	14,57	8,56	14,3	476,6
20000	2048	100	121,162 $\pm$ 0,462	10 274,472 $\pm$ 42,107	71,15	42,38	14,3	476,6
200000	2048	5	390,059 $\pm$ 0,512	103 803,161 $\pm$ 49,981	45,29	27,30	102,4	4 726,1
200000	2048	20	386,061 $\pm$ 0,335	103 787,889 $\pm$ 60,275	176,99	108,27	102,5	4 726,0

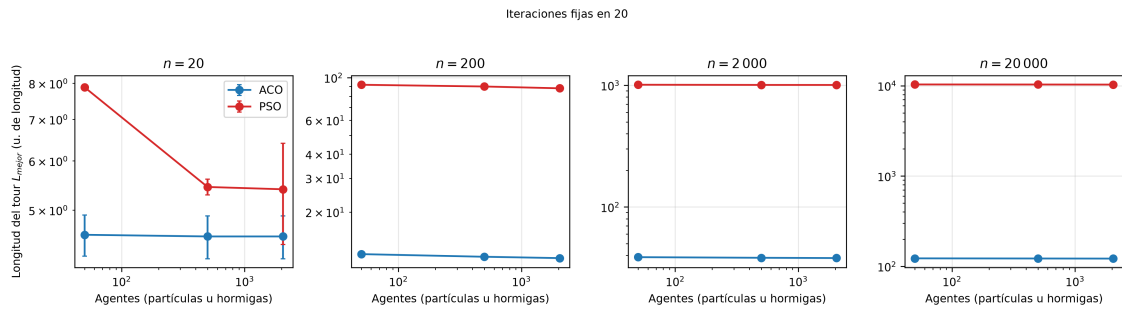


Figura 7: L_{mejor} contra agentes, con 20 iteraciones.

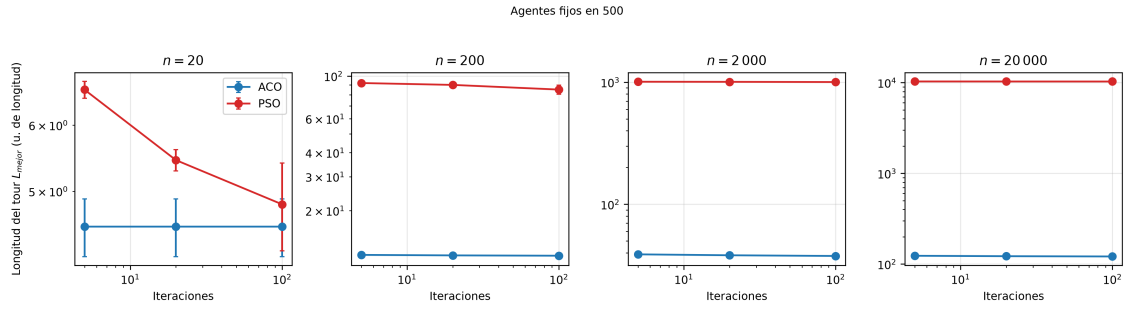


Figura 8: L_{mejor} contra iteraciones, con 500 agentes.

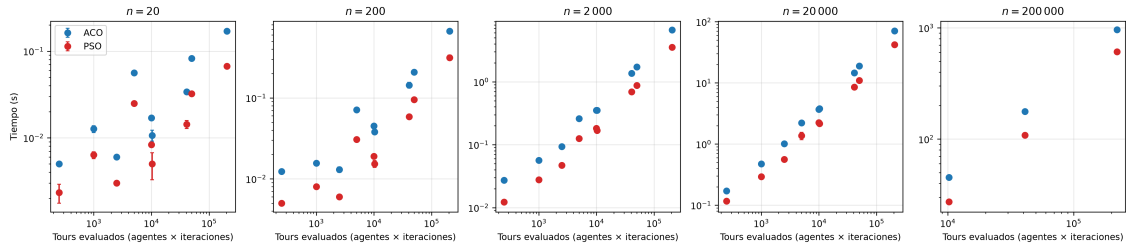


Figura 9: Tiempo contra tours evaluados (agentes $\times$ iteraciones).

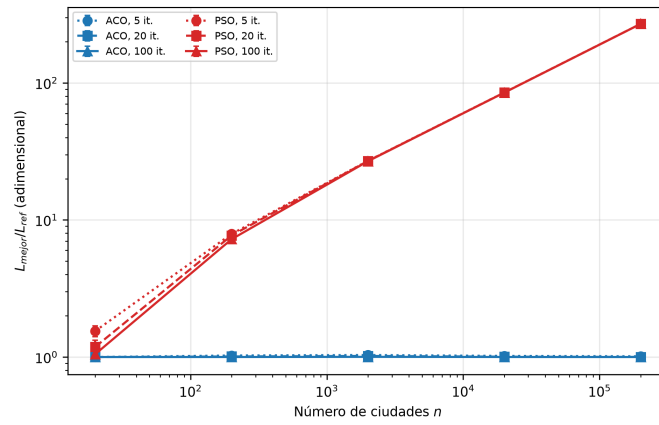


Figura 10: L_{mejor}/L_{ref} contra n con 2048 agentes (L_{ref} : óptimo de Held-Karp en $n = 20$; en los demás n , la mejor longitud medida).